Advanced Reasoning and Test-Time Compute

Search, Reward Models, Exact Verification, Self-Correction, and Planning

Naeemullah Khan
naeemullah.khan@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

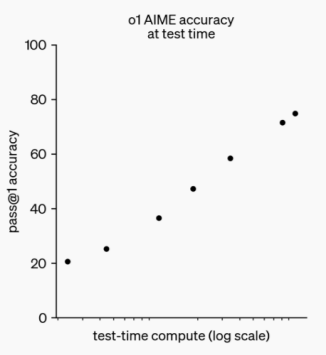
October 11, 2026

Table of Contents

1. Motivation
2. Learning Outcomes
3. Scaling Reasoning with Test-Time Compute
4. The Evolution of Chain-of-Thought
5. Search-Based Reasoning
6. Process and Outcome Reward Models
7. Program-of-Thought and Code-Assisted Reasoning
8. Mathematical Reasoning and Verification
9. Iterative Reasoning and Self-Correction
10. Deliberative Reasoning and Planning
11. Neural Search and Reasoning Optimisation
12. Reasoning-Model Paradigms in 2026
13. Summary

Same Weights, 74% or 93%

- OpenAI's o1 on AIME 2024 mathematics problems. GPT-4o solves 12%. o1 solves **74% to 93%, depending on inference settings**.
- The ladder: 74% with one sample, 83% with a vote over 64 samples, 93% when a learned scorer re-ranks 1,000 samples.
- Nothing in the model changed between those three numbers. What changed is how many candidates were generated and what chose among them.
- One reading: test-time scaling is "a new axis for scaling compute". The plot on the right is linear in the *logarithm* of compute, so each constant gain costs a multiple.



Range and GPT-4o baseline: Wolfe, "Demystifying Reasoning Models" (Feb 2025). Per-setting ladder: OpenAI, "Learning to reason with LLMs" (Sep 2024). Quote: Hoogland, "o1: A Technical Primer" (Dec 2024). Plot: OpenAI's o1 launch figure as reproduced in Xiang et al., arXiv:2501.04682, Figure 9.

One Idea, Told Ten Times

- Every method today has the same two parts: **generate** candidates, then **select or correct** them.
- Generation is cheap to scale. The question is always what does the selecting.

The checker is	Examples	What happens as compute grows
Exact	unit tests, Lean, a classical planner, a robust answer grader	coverage turns into accuracy. Search works
A model	a reward model, an LLM judge, the model's own critique	gains plateau, over-optimise, or get hacked

- The 2025 to 2026 shift: the industry moved the checker *inside training* and left the user an **effort dial**.

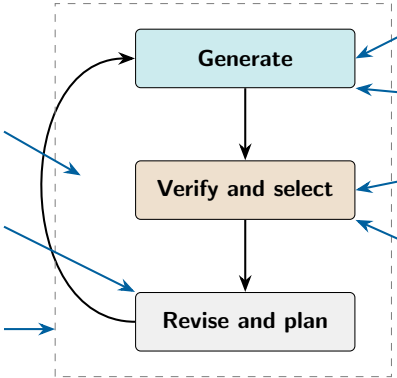
Compute spent at inference is only as useful as the signal that selects or corrects what it produces.

Today's Map: One Loop, Ten Questions

7. Can a model fix itself?
Self-correction and its failures

8. Can it plan?
Planning with a verifier

10. Where do models stand?
Reasoning paradigms in 2026



9. can the whole loop be learned?

1. How does thinking scale?
Test-time compute

2, 3. How to structure it?
Chains, trees, graphs, search

4. What scores a step?
Process and outcome rewards

5, 6. Can the check be exact?
Code, symbolic tools, Lean

Learning Outcomes

By the end of this session, you should be able to:

- **Explain** why test-time compute scales log-linearly, and why the verifier bounds it.
- **Compare** Least-to-Most and Graph-of-Thought with chains and trees, and rank selectors beyond the vote.
- **Analyse** best-of-N, beam-style search and MCTS, and when search beats sampling.
- **Contrast** process and outcome reward models, at inference and inside RL.
- **Describe** how code execution, symbolic engines and Lean make a check exact.
- **Diagnose** when Self-Refine, Reflexion and related loops help, and when they fail.
- **Evaluate** planning claims and the 2026 reasoning models from public evidence.

Prerequisite knowledge: chain-of-thought prompting, self-consistency, Tree-of-Thought, PAL, pass@k; outcome and process reward models, reward over-optimisation, RL with verifiable rewards, GRPO; DeepSeek-R1 and R1-Zero's "aha moment"; overthinking and the Illusion of Thinking; unfaithful and monitorable chain-of-thought traces; compute-optimal and over-trained scaling.

Advanced Reasoning: **Test-Time Compute**

If a model may think for longer, how much does it gain, and what decides the limit?

Why Intermediate Tokens Help, and Four Ways to Spend Them

- **Theory.** A constant-size transformer can solve any problem solvable by a boolean circuit of size T by generating $O(T)$ intermediate tokens. Tokens buy *serial computation*.
- **The catch.** Errors compound. With a per-step error ϵ, a chain of T steps is right with probability $(1-\epsilon)^T$. For $\epsilon = 0.01$ and $T = 128$ that is 0.276.
- So a long generation needs *checking*. A meta-generator $y \sim \mathcal{G}(y \mid x, g_1, \ldots, g_G, \phi)$ wraps one or more generators g_i in a strategy:

Pattern	Examples	What it adds
Chained	chain-of-thought, decomposition	depth, but no exploration of the output space
Parallel	best-of-N, voting	width
Tree search	beam search, REBASE, MCTS	width, guided step by step
Refinement	self-correction, trained correctors	depth, guided by feedback

Li et al. (ICLR 2024), as presented in Zhou, "LLM Reasoning," Stanford CS25 (Apr 2025). Welleck, CMU 11-711 "Decoding Algorithms" (Sep 2025). Welleck et al., "From Decoding to Meta-Generation," arXiv:2406.16838, and NeurIPS 2024 tutorial slides.

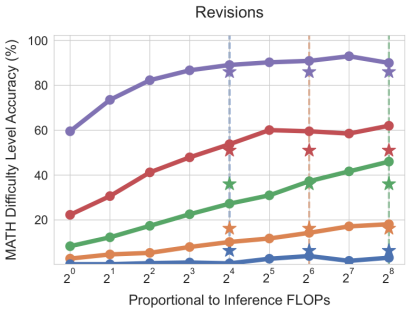
Coverage Is Not Accuracy

- Sample k answers. **Coverage** $c(k)$ is the fraction of problems where at least one is right, measured as pass@k. It fits
$$c(k) \approx \exp\left(a\, k^{b}\right)$$
- Coverage grows roughly **log-linearly over four orders of magnitude**.
  - SWE-bench Lite, DeepSeek-Coder-V2: 15.9% with 1 sample, **56% with 250**. Single-sample state of the art then: 43%.
- The selection gap. Without an automatic verifier, majority voting and reward models plateau after a few hundred samples.

2026, open-ended domains (medicine, law, writing): "the candidate pool is not the bottleneck, choosing from it is." Reward models correlate with true quality at only $\rho \approx 0.12$.

Brown et al., "Large Language Monkeys," arXiv:2407.21787. Romano et al., "Test-Time Scaling in the Wild," arXiv:2608.18931.

How Should a Budget Be Spent? It Depends on Difficulty



Lines: a small model with more inference compute, from easiest (top) to hardest questions. Stars: a larger pre-trained model.

- **Compute-optimal strategy:** per question, pick the method and settings that maximise expected accuracy for the budget. It matches best-of-N with about **4× less compute**.
- Easy questions prefer sequential revision, harder ones a mix with parallel search. On the hardest bin nothing helps.
- **Against parameters.** Let R be inference tokens over pre-training tokens. If $R \ll 1$, test-time compute wins on easy and medium questions. If $R \gg 1$, pre-train more.

"Beats a 14× larger model" holds only where the small model already has non-trivial success.

Snell et al., "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters," arXiv:2408.03314, Figure 9 (left panel).

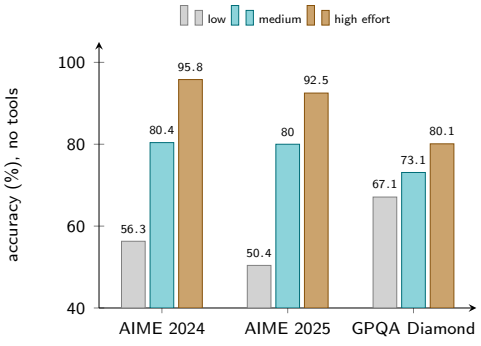
Inference Has Its Own Scaling Laws

- **Small models can be compute-optimal.** Llemma-7B with tree search consistently beats Llemma-34B on MATH. Larger models win only after smaller ones saturate.
- **No strategy dominates.** A replication over 30B generated tokens, 8 open models and 4 datasets finds no single best test-time strategy.
- **2026: training and inference are one budget.** Train-to-Test laws optimise model size, training tokens and inference samples together. Once inference cost is counted, the optimum shifts "radically into the overtraining regime".

Three steps. Chinchilla optimises training FLOPs. Wu et al. optimise inference FLOPs for a given model. Train-to-Test optimises both, and lands on over-trained small models.

Wu et al., "Inference Scaling Laws," arXiv:2408.00724. Agarwal et al., "The Art of Scaling Test-Time Compute for LLMs," arXiv:2512.02008. Roberts et al., "Test-Time Scaling Makes Overtraining Compute-Optimal," arXiv:2604.01411.

Reinforcement Learning Makes Long Chains Productive



gpt-oss-120b at three reasoning-effort levels

- Prompting a model to think longer does little. **Training it with outcome rewards** is what makes length pay.
- **DeepSeek-R1-Zero,** AIME 2024: 77.9% pass@1, 86.7% with a vote over 16. Parallel scaling still stacks on top.
- **Kimi k1.5:** long-context RL reaches AIME 77.5.
- **Qwen3, Gemini 2.5, gpt-oss:** smooth gains as the thinking budget grows.

OpenAI, gpt-oss model card, arXiv:2508.10925. DeepSeek-AI, "DeepSeek-R1," arXiv:2501.12948 (v2 numbers). Kimi Team, "Kimi k1.5," arXiv:2501.12599. Qwen3 Technical Report, arXiv:2505.09388.

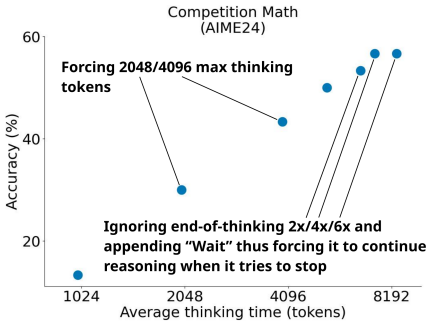
But Longer Is Not Always Better

- **Within a trained model, length is a noisy signal.** For QwQ, R1 and LIMO, correct solutions are often *shorter* than incorrect ones.
- **An inverted U.** Accuracy peaks at an optimal chain length. It grows with difficulty and *shrinks* with capability.
- **Attractors (2026).** 53.8% of sequential scaling runs enter, within four iterations, a set of answers they never leave. Mixing models lowers that rate by 21.2 points.
- **Inverse scaling.** With longer reasoning, nine models get worse on some tasks.
- **Knowledge tasks.** More thinking often adds hallucinations. It cannot add facts.

Chain length is an allocation problem. Too long wastes tokens and can lower accuracy; switching thoughts too often fails as well.

Zeng et al., arXiv:2502.12215. Wu et al., "When More is Less," arXiv:2502.07266. Rance et al., arXiv:2609.39632. Gema, Hägele et al., arXiv:2507.14417. Knowledge tasks: arXiv:2509.06861. Switching thoughts: Wang et al., arXiv:2501.18585.

Controlling the Budget: From Forcing to Training



- **s1: budget forcing at decode time.** To cap thinking, append the end-of-thinking delimiter. To extend it, suppress the delimiter and append "Wait".
  - AIME24 rises from 50% to 57%, then "eventually flattens out".
- **L1: train the length in.** The prompt says "Think for N tokens", and the reward penalises missing it:

$$r = \mathbb{1}(y = y_{\text{gold}}) - \alpha\,|n_{\text{gold}} - n_y|$$

  - 20 to 25 points above budget forcing at equal length. Mean length error about 3%.
- Trained control beats decode-time intervention. Production systems took the same road.

Muennighoff et al., "s1: Simple test-time scaling," arXiv:2501.19393, Figure 1. Aggarwal and Welleck, "L1: Controlling How Long A Reasoning Model Thinks With RL," arXiv:2503.04697.

From Token Budgets to Effort Dials (October 2026)

Provider	Control	Note
Anthropic	adaptive thinking plus effort: low to max	token budgets are rejected on Claude 4.7 and later
OpenAI	`reasoning.effort`: none to max	reasoning tokens are hidden but billed as output
Google	`thinking_level`: minimal to high	thought tokens count against the output limit
xAI	effort: low to xhigh	cannot be switched off. On the multi-agent model, effort sets *how many agents collaborate*
Open weights	Qwen3 `/think`, gpt-oss "Reasoning: low"	DeepSeek-V4.1-Flash: effort from 1 to 100

- Three generations of control: decode-time forcing, trained length conditioning, then a **coarse dial with the model deciding** how much to think.

Anthropic, extended thinking and effort documentation. OpenAI, reasoning models guide. Google, Gemini API thinking documentation. xAI, reasoning guide. Qwen3 blog (Apr 2025). DeepSeek-V4.1-Flash release note (Sep 2026). All accessed 3 Oct 2026.

Test-Time Compute: What to Remember

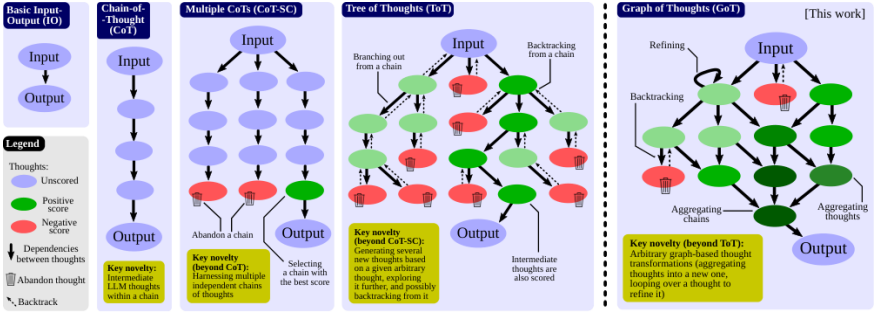
- Test-time compute is **generation plus selection**. Coverage is the upper bound. The verifier decides how much of it you keep.
- Gains are **log-linear**, depend on difficulty, and are bounded by the base model.
- **Outcome-reward RL** is what turned longer chains into better answers. Within a trained model, length alone is a noisy signal.
- The interface settled on **effort dials**. The research question moved from "does it scale?" to "how do we allocate, verify and bound it?"

Next. Tokens can be spent in one chain, many chains, a tree or a graph. Which structure, and does the choice still matter?

Advanced Reasoning: **Chain-of-Thought Evolution**

One chain, many chains, a tree or a graph: what does each structure buy?

The Family in One Picture



- Read the family as shapes: a **path** (CoT), a **bundle** (Self-Consistency), a **planned sequence** (Least-to-Most), a **tree** (ToT), a **graph** (GoT).
- Each step adds one operation: vote, decompose, backtrack, aggregate. Each also needs something to *score* thoughts.

Besta et al., "Graph of Thoughts: Solving Elaborate Problems with Large Language Models," AAAI 2024, arXiv:2308.09687, Figure 1.

A Ladder of Selectors

- A vote counts answers. What if the output is free text, or the majority is wrong?

Selector	How it chooses	Evidence
Self-Consistency	unweighted vote on the final answer	GSM8K with PaLM 2-L: greedy 85.7, vote 90.4
Universal Self-Consistency	the LLM reads all candidates and picks "the most consistent"	90.2 on the same task. Also works on summaries and SQL, with no execution
Verifier-weighted vote (DIVERSE)	a trained verifier weights each vote, step-aware	GSM8K 74.4 to 83.2
Exact verifier	tests, execution, a proof checker	the plateau disappears. Coverage becomes accuracy

Chen et al., "Universal Self-Consistency for LLM Generation," arXiv:2311.17311, Table 1. Li et al., "Making LLMs Better Reasoners with Step-Aware Verifier," arXiv:2206.02336. Brown et al., arXiv:2407.21787.

Least-to-Most: Chain-of-Thought with a Planner

- **Problem:** CoT fails on problems *harder than its demonstrations*.
- **Method:** two prompted stages, no training. *Decompose* into easier subproblems, then *solve* them in order, each answer feeding the next.
- **SCAN, length split:** 99.7% with 14 exemplars. CoT reaches 16.2%.
- **The limit:** decomposition prompts transfer poorly across domains. Almost every failed GSM8K problem becomes solvable with a hand-written decomposition.

Last-letter concatenation, accuracy (%)

List length	4	8	12
Standard prompt	0.0	0.0	0.0
Chain-of-thought	84.2	50.2	31.8
Least-to-Most	94.0	83.0	74.0

Its value is **easy-to-hard generalisation**, not average accuracy. On GSM8K the gain appears only at 5 or more steps: 45.2 against 39.1.

Zhou et al., "Least-to-Most Prompting Enables Complex Reasoning in Large Language Models," ICLR 2023, arXiv:2205.10625, Tables 4, 8 and 12. Model: code-davinci-002.

From Tree to Graph of Thoughts

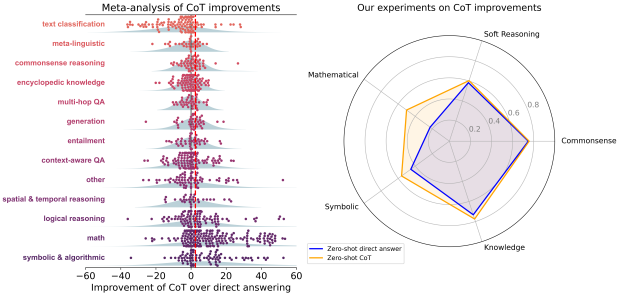
- **Why a graph?** A tree can branch and backtrack, but it cannot *merge* two good partial results.
- **Formalism.** Thoughts are vertices. Three transformations: *generate* (as in ToT), *aggregate* k thoughts into one, *refine* a thought in a loop.

Scheme, at total cost $\Theta(N)$	**Latency**	**Volume of a thought**
Chain-of-thought	N	N
Self-Consistency, k chains	N/k	N/k
Tree-of-Thought, k-ary tree	$\log_k N$	$O(\log_k N)$
Graph-of-Thought	$\log_k N$	N

- *Volume*: how many earlier thoughts can influence a thought. Only the graph gets low latency *and* high volume.
- Sorting 128 numbers with GPT-3.5: median error 62% lower than ToT, at more than 31% lower cost.

Besta et al., arXiv:2308.09687, Section 3 and Table 2.

When Does Chain-of-Thought Help at All?



- A meta-analysis of 1,218 comparisons from 110 papers, plus 20 datasets on 14 models.
- Average gain: **maths +12.3, symbolic +14.2, logic +6.9**. Everything else: +0.7.
- And where CoT helps, a planner plus a real *tool* solver beats it. CoT is mostly symbolic execution, done badly.

Sprague et al., "To CoT or not to CoT?" ICLR 2025, arXiv:2409.12183, Figure 1. This studies *prompted* CoT on 2024 models.

What Reasoning Models Made Obsolete

Status in 2026	Technique and evidence
Superseded	*"Think step by step."* On GPQA Diamond it changes o3-mini and o4-mini by about +0.03 and costs 20 to 80% more time. On Gemini Flash 2.5 it hurts
Superseded	*Few-shot CoT exemplars.* "Few-shot prompting consistently degrades" R1. Five shots lowered o1-preview on MedQA
Still useful	*Voting and ensembling*, and explicit instructions on format and constraints
Moved inside the model	*Decomposition and search.* With thought and answer sampled together, "the model can learn a search algorithm"

- These structures were hand-designed. RL-trained models learn the structure inside one chain.

Meincke et al., "The Decreasing Value of Chain of Thought in Prompting," arXiv:2506.07142. DeepSeek-AI, arXiv:2501.12948. Nori et al., "From Medprompt to o1," arXiv:2411.03590. Welleck, CMU 11-711 (Nov 2025).

Chain-of-Thought Evolution: What to Remember

- The family is a sequence of shapes: **path, bundle, planned sequence, tree, graph**.
- Selectors climb a ladder: **vote, model pick, trained verifier, exact verifier**. Voting survives into the reasoning-model era. Prompted and few-shot CoT do not.
- Least-to-Most buys **length generalisation**. Graph-of-Thought buys **aggregation**.
- Prompted CoT helps mainly on **maths and symbolic** tasks, where a tool does better still.

Zhou's four-line summary (2025)

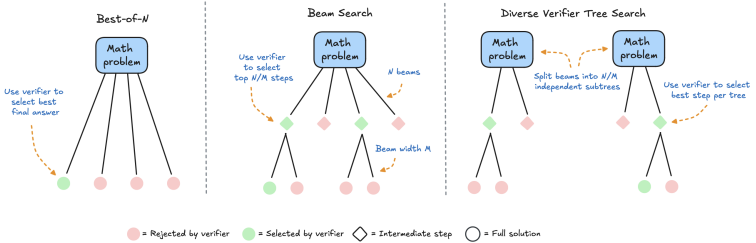
Reasoning beats no reasoning. RL fine-tuning beats SFT. Aggregating several answers beats one answer. Retrieval plus reasoning beats reasoning alone.

Next. Trees and graphs need a rule for which thought to expand. That rule is *search*, and it needs a scorer.

Advanced Reasoning: **Search-Based Reasoning**

Given a scorer, is it better to sample many answers or to search step by step?

Three Ways to Use a Verifier



- **Best-of-N:** score each full solution once, keep the best.
- **Beam search:** score every *step* with a process reward model. Keep the top N/M, expand each M ways.
- **DVTS:** split the budget into N/M independent subtrees. Less exploitation, more diversity.

Figure: Beeching, Tunstall and Rush (Hugging Face), "Scaling test-time compute with open models" (2024). Beam search with a process reward model follows Snell et al., arXiv:2408.03314.

Aggregation, and Why It Saturates

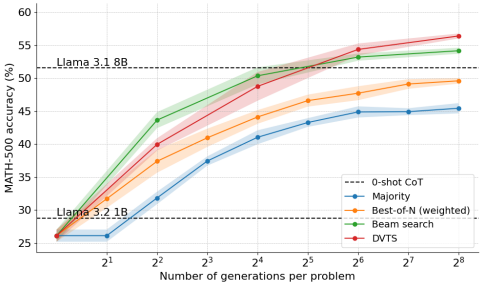
- With samples $y^{(1)}, \ldots, y^{(N)}$, answers a_i and a scorer v:

$$\underbrace{\arg\max_{y^{(i)}} v\big(y^{(i)}\big)}_{\text{best-of-N}} \qquad \underbrace{\arg\max_{a} \sum_{i} \mathbb{1}[a_i = a]}_{\text{majority vote}} \qquad \underbrace{\arg\max_{a} \sum_{i} v\big(y^{(i)}\big)\, \mathbb{1}[a_i = a]}_{\text{weighted vote}}$$

- **A limit theorem.** As $N \to \infty$, voting accuracy converges to a value fixed by the generator g and the scorer v. To pass it, improve v or g.
- **Best-of-N is optimisation against a proxy**: as N grows, the *true* reward rises, peaks, then falls, because a large enough N selects what fools the scorer.
- **A ceiling from false positives.** Resampling cannot lower a verifier's false-positive rate. The optimal number of attempts is then rarely above 10.

Welleck, CMU 11-711 (Nov 2025). Wu et al., arXiv:2408.00724, Theorem 2. Gao, Schulman and Hilton, "Scaling Laws for Reward Model Overoptimization," arXiv:2210.10760. Stroebl, Kapoor and Narayanan, arXiv:2411.17501 (abstract).

Search Is Not Uniformly Better than Sampling

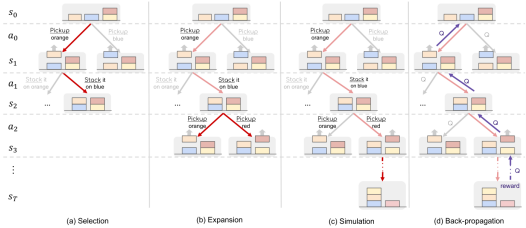


Llama 3.2 1B on MATH-500, with an 8B process reward model

- **At small budgets, beam search wins.** Beam with 4 generations matches best-of-N with 16. Beam with 16 matches best-of-N with 256.
- **At large budgets on easy problems, it loses.** The beam over-exploits the reward model. DVTS restores diversity.
- **REBASE** spreads the budget by score, $W_j \propto e^{R(n_j)/T_b}$, with no rollouts. About $7\times$ less compute than sampling.
- A 3B model with the right strategy per difficulty passes Llama 3.1 70B, with about $22\times$ fewer parameters.

Beeching, Tunstall and Rush (Hugging Face), "Scaling test-time compute with open models" (2024). Snell et al., arXiv:2408.03314. Wu et al., arXiv:2408.00724.

Monte Carlo Tree Search on Text



Blocksworld steps	2	4	6
CoT, LLaMA-33B	0.17	0.02	0.00
CoT, GPT-4	0.50	0.63	0.40
RAP, 20 iterations	1.00	0.88	0.42

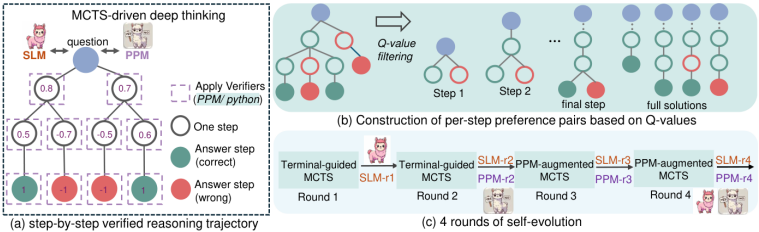
- **The mapping.** A node is a partial trace, its state defined per task. An action is the next step. The value comes from rollouts, self-evaluation or a learned model.
- **Selection** balances value against novelty:

$$a^* = \arg\max_a \left[Q(s,a) + w\sqrt{\frac{\ln N(s)}{N(c(s,a))}} \right]$$

- **RAP:** one frozen LLM proposes actions and, as the *world model*, predicts the next state.
- A small model with search beats GPT-4 with CoT: 64% average.

Hao et al., "Reasoning with Language Model is Planning with World Model," EMNLP 2023, arXiv:2305.14992, Eq. 1 and Table 1. Rush and Ritter, "Speculations on Test-Time Scaling" (2024).

rStar-Math: Search as a Data Generator



(a) step-by-step verified reasoning trajectory

(b) Construction of per-step preference pairs based on Q-values

(c) 4 rounds of self-evolution

- **Each node is text plus Python.** A step whose code fails is dropped. Four rounds of self-evolution improve policy and scorer together.
- **A preference model, not a value regressor.** Q-values from rollouts are noisy, so train on pairs: $\mathcal{L}_{\text{ppm}} = -\frac{1}{4}\,\mathbb{E}\big[\log \sigma\big(r_\theta(x, y^{\text{pos}}) - r_\theta(x, y^{\text{neg}})\big)\big]$.
- Qwen2.5-Math-7B on MATH: 58.8% to **90.0%**. AIME 2024: 8 of 15. The model also learned to abandon a flawed approach and backtrack, without being trained to.

Guan et al. (Microsoft Research Asia), "rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking," arXiv:2501.04519.

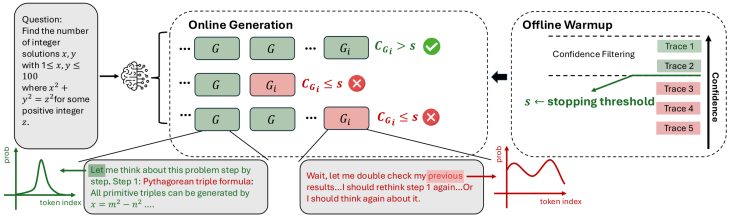
Why Frontier Labs Skipped Tree Search

- In late 2024, AlphaZero-style search was a leading guess about how o1 was built. Then came the open recipes.
- **DeepSeek-R1, "Unsuccessful Attempts":**
  - token generation has an exponentially larger search space than chess or Go
  - capping the expansions per node traps the search in local optima
- **Kimi k1.5:** planning can be *approximated inside the context*. Thought tokens play the role of a planning budget.

Where explicit search survives: where a terminal check is cheap and exact (code, formal maths), and as a *generator of training data*.

DeepSeek-AI, arXiv:2501.12948, Appendix G.2 (v2). Kimi Team, arXiv:2501.12599, Sections 2.3 and 4. Rush and Ritter, "Speculations on Test-Time Scaling" (2024).

Better Selectors for Parallel Samples (2025)



- **DeepConf: use the model's own confidence.** Score a trace by its *weakest* window. Warm up on 16 traces, then **stop any trace that drops below the threshold**.
- gpt-oss-120b, AIME 2025, budget 512: majority vote 97.1%. DeepConf 97.9% with **84.7% fewer tokens**.
- **GenSelect: let a reasoning model choose** among all candidates in one prompt. QwQ-32B, 64 candidates: majority 68.4%, GenSelect **73.4%**.

Fu et al. (Meta AI, UCSD), "Deep Think with Confidence," arXiv:2508.15260. Toshniwal et al. (NVIDIA), "GenSelect," arXiv:2507.17797.

Search: What to Remember

- Best-of-N, beam search and MCTS differ in **where the scorer is applied**: once per answer, or once per step.
- Step-level search wins at **small budgets and on hard problems**. It loses when it over-exploits the scorer.
- Voting and re-ranking **saturate**. The limit is set by the generator and the scorer, not by N.
- Explicit tree search did not become the route to reasoning models. It became a **data generator**, and a tool where checks are exact.

Next. Every method here leaned on a scorer of steps or of answers. How are such scorers built, and can they be trusted?

Advanced Reasoning: **Process vs Outcome Reward Models**

Should a scorer judge the final answer or every step, and what happens when a policy trains against it?

Four Kinds of Scorer

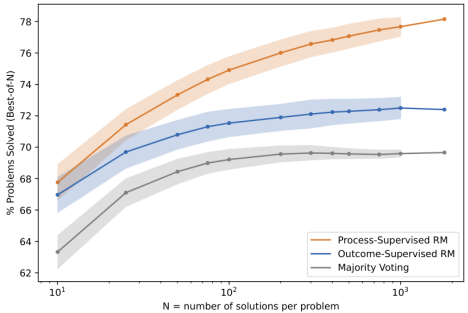
Model	Predicts	Trained with
Preference reward model (RLHF)	one scalar per sequence	pairwise loss on human comparisons
Outcome reward model (ORM)	is the final answer correct?	binary cross-entropy on the outcome label
Process reward model (PRM)	is each *step* correct?	per-step labels at step delimiters
Value function	expected remaining return	regression on on-policy rollouts

$$\mathcal{L}_{\text{ORM}} = y_s \log r_s + (1 - y_s) \log(1 - r_s) \qquad \mathcal{L}_{\text{PRM}} = \sum_{i=1}^{K} y_{s_i} \log r_{s_i} + (1 - y_{s_i}) \log(1 - r_{s_i})$$

- A PRM gives K scores per solution. To rank solutions, reduce them to one: **product**, **minimum** or **last step**. The best choice depends on where the labels came from.

Lambert, RLHF book, reward modelling chapter (accessed Oct 2026). Losses as in Wang et al., "Math-Shepherd," arXiv:2312.08935.

The Classic Result: Verify Step by Step



- **PRM800K:** about 800,000 human labels on the steps of 75,000 solutions to 12,000 MATH problems.
- **Best-of-1860 on MATH:** PRM 78.2%, ORM 72.4%, majority vote 69.6%. The PRM curve is still rising where the others are flat.
- **Out of distribution** (recent AP and AMC exams): 72.9, 63.8 and 61.3.
- **Active learning** made labelling about 2.6× more efficient. Annotators saw mostly "convincing wrong-answer" solutions.

Lightman et al. (OpenAI), "Let's Verify Step by Step," arXiv:2305.20050, Figure 3 and Table 1.

Step Labels Without Humans

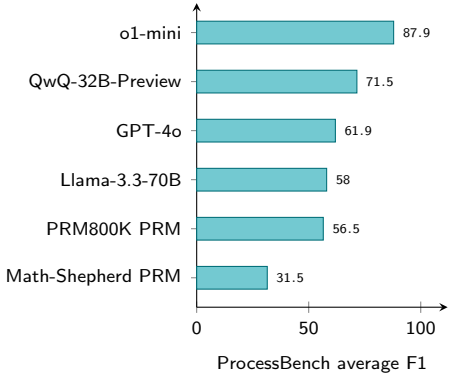
- Human step labels do not scale. **Monte Carlo labels:** from step s_i, sample N completions and check their answers a_j against the truth a^*:

$$y_{s_i}^{\text{hard}} = \mathbb{1}\left[\exists j : \; a_j = a^*\right] \qquad y_{s_i}^{\text{soft}} = \frac{1}{N}\sum_{j=1}^{N} \mathbb{1}[a_j = a^*]$$

- What this actually measures. Not whether the step is valid, but whether *this completer* can finish from here. It is a value estimate.
- **Implicit PRM:** no step labels at all. Train on outcomes with $r_\theta(\mathbf{y}) = \beta \log \frac{\pi_\theta(\mathbf{y})}{\pi_{\text{ref}}(\mathbf{y})}$. Its partial sums are Q-values, and their increments are step rewards.
- **PRIME** trains policy and implicit PRM together, online. A *static* PRM decays from about 75% to 45% accuracy as the policy moves.

Wang et al., "Math-Shepherd," arXiv:2312.08935. Yuan et al., "Free Process Rewards without Process Labels," arXiv:2412.01981. Cui et al., "PRIME," arXiv:2502.01456.

Is Your PRM a Process Model, or an Outcome Judge in Disguise?



Task: find the earliest wrong step in 3,400 solutions

- Best-of-N rewards a PRM for guessing the *outcome*. **ProcessBench** asks the real question: where is the first error?
- Trained PRMs trail prompted critics. By label source: human 56.5, LLM judge 46.5, Monte Carlo 40.1.
- **Process-to-outcome drift.** For several PRMs, over 40% of minimum scores fall on the *final* step.
- That explains the reductions. Monte Carlo PRMs work best with "last step". Human-labelled ones with product or minimum.

Zheng et al. (Qwen), "ProcessBench," arXiv:2412.06559. Zhang et al. (Qwen), "The Lessons of Developing Process Reward Models in Mathematical Reasoning," arXiv:2501.07301, Tables 4 and 5.

Verification as Generation

- A discriminative PRM is a classifier with fixed compute. Why not let the verifier *think*?
- **Generative verifier (GenRM).** Ask "Is the answer correct?" and read the probability of "Yes". With verification chains $v^{(i)}$, average over K of them:

$$r_{\mathrm{MajV@}K}(x, y) = \frac{1}{K} \sum_{i=1}^{K} p_\theta\left(\mathrm{Yes} \mid x, y, v_{\mathrm{CoT}}^{(i)}\right)$$

  Accuracy rises with K. **The verifier has its own test-time scaling.**

- **ThinkPRM.** Fine-tune a reasoning model on about 1,000 verification chains. It beats discriminative PRMs trained on 712,000 labels, about 100× more.
- **In production recipes.** Kimi k1.5 compared a value-head reward model with a chain-of-thought one on spot checks: 84.4% against 98.5%. The second went into RL.

Zhang et al., "Generative Verifiers," ICLR 2025, arXiv:2408.15240. Khalifa et al., "Process Reward Models That Think," arXiv:2504.16828. Kimi Team, arXiv:2501.12599.

Inside RL, Outcome Rewards Won

- A denser reward should help. Yet DeepSeek-R1 and Kimi k1.5 trained without a PRM. What happens when one is used?

Use of the process signal	What happened
Added to the reward	entropy collapses, responses grow and repeat. Ends *below* plain GRPO (47.3 against 49.9)
Policy trained against a PRM	PRM reward above 0.9 while true accuracy stays below 4%
As a data filter (PROF)	drop samples where process and outcome disagree. 51.7 against 49.9
Online and implicit (PRIME)	the scorer moves with the policy

- **Theory agrees.** Outcome supervision is "no more statistically difficult" than process supervision, and the advantage function of any policy is an optimal PRM.

Under mild assumptions, GRPO with an outcome reward *already is* a Monte Carlo PRM.

DeepSeek-AI, arXiv:2501.12948. Kimi Team, arXiv:2501.12599. Ye et al. (PROF), arXiv:2509.03403, Table 2. Tiwari et al., "Reward Under Attack," arXiv:2603.06621. Jia, Rakhlin, Xie, "Do We Need to Verify Step by Step?" arXiv:2502.10581. Sullivan and Koller, "GRPO is Secretly a Process Reward Model," arXiv:2509.21154.

Reward Models: What to Remember

- **For selection at inference**, process models beat outcome models and votes. But best-of-N alone is a biased way to measure a PRM.
- **Monte Carlo step labels estimate a value**, not logical validity. Check which one your PRM learned.
- **Generative verifiers** are replacing discriminative ones. They need far fewer labels and scale with compute.
- **As an RL reward**, a static learned PRM gets hacked. Verifiable outcome rewards stay the anchor.

State of play, October 2026

No public source shows a frontier lab using a learned PRM as the RL reward of a production model.

Next. A learned scorer can always be fooled. An interpreter cannot. What if part of the reasoning is *executed* instead of judged?

Advanced Reasoning: **Code-Assisted Reasoning**

What changes when part of the reasoning is executed by an interpreter?

Program of Thoughts: Separate Reasoning from Computation

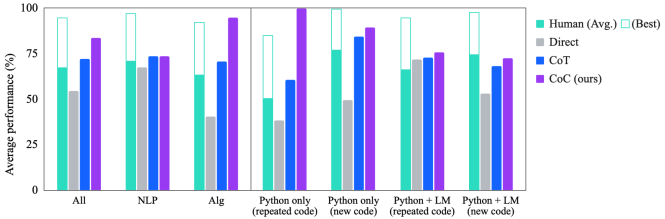
- **Program of Thoughts (PoT)** appeared alongside PAL in 2022. Zero-shot, it beats CoT on every benchmark tested (right). It is within 0.4 points of PAL on GSM8K.
- **What lasts is the ablation.** GSM8K: full method 71.6. Variables renamed to a, b, c: 60.2. One direct equation: 45.8. Multi-step programs and *semantic binding* do the work.
- **And the error analysis.** Of 198 failures on one benchmark, 47% are *value grounding*. Code does not fix that.

Zero-shot accuracy (%)

Benchmark	CoT	PoT
GSM8K	40.5	57.0
AQuA	31.9	43.9
SVAMP	63.7	70.8
TabMWP	53.5	66.5
MultiArith	79.3	92.2
Average	53.7	66.1

Chen, Ma, Wang and Cohen, "Program of Thoughts Prompting," TMLR 2023, arXiv:2211.12588, Tables 3, 5 and 6. Gao et al., "PAL: Program-aided Language Models," arXiv:2211.10435.

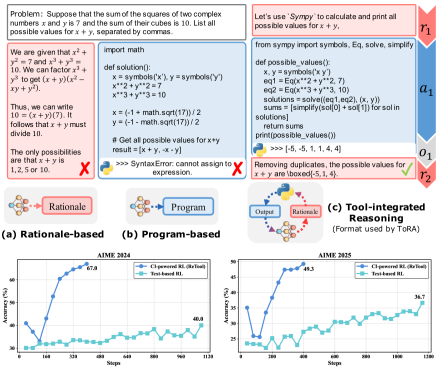
Chain of Code: When a Line Cannot Run, Simulate It



- **Problem:** many reasoning steps are semantic. No library implements `is_sarcastic(s)`.
- **Idea:** write code that mixes real code and pseudocode. Run it line by line. When a line fails, the language model *simulates* its effect on the program state.
- BIG-Bench Hard: **84%**, against 72% for CoT and 68% for the average human rater.
- The ablation that matters: **Python alone gives 48%**. The language model alone tracking state gives 63%.

Li et al., "Chain of Code," ICML 2024, arXiv:2312.04474, Figure 4 and Tables 1 and 2. Model: text-davinci-003.

From Prompted Programs to Trained Tool Use



- **ToRA (supervised).** Interleave rationale r_i, program a_i and interpreter output o_i: $\tau = r_1 a_1 o_1 \ldots r_{n-1} a_{n-1} o_{n-1} r_n$. It beats program-only by 6.7 points.
- **ReTool (RL).** Run a real sandbox *inside* the rollouts. Two details recur in later systems:
  - interpreter output is masked out of the loss
  - the reward is the outcome only: $+1$ or -1
- AIME 2024 with a 32B model: **67.0% in 400 steps**. Text-only RL: 40.0% in 1,080 steps. Responses get about 40% shorter.
- The model learns *when* to call code, and to repair it from error messages.

Gou et al., "ToRA," ICLR 2024, arXiv:2309.17452. Feng et al. (ByteDance Seed), "ReTool," arXiv:2504.11536, Figure 1 and Table 1.

When Code Hurts

- A study of 7 methods, 14 tasks and 6 models asks: is code always the better choice?
- **Text beats code on planning tasks.** Correct code would have to combine constraint checking, optimisation and simulation.
- **Inverse scaling.** On Game of 24, GPT-3.5 and GPT-4o-mini reach 100% because they always write code. GPT-4o is overconfident in text and skips the code on medium questions.
- **With feedback over several turns,** code methods keep improving. Text-only self-reflection degrades.

Code for *execution*: exact computation, enumeration, checking. Language for *grounding and planning*. The best systems interleave both.

Chen et al., "Steering Large Language Models between Code Execution and Textual Reasoning," ICLR 2025, arXiv:2410.03524. Li et al., arXiv:2312.04474. Chen et al., arXiv:2211.12588.

Code-Assisted Reasoning: What to Remember

- Code separates **what to compute** from **computing it**. The interpreter is an exact executor.
- The gain comes from **multi-step programs with meaningful names**, and from interleaving code with language.
- The lineage: one program run once, then a simulated interpreter, then supervised tool use, then tool use **learned by outcome RL** and built into o3 and o4-mini.
- Code does not fix **grounding errors**, and it can make planning worse.

A common misconception

"A code interpreter always helps with mathematics." It helps with computation. It does not check that the reasoning around the computation is sound.

Next. Execution checks arithmetic. What would it take to check a whole *argument*? That is the job of mathematical verification.

Advanced Reasoning: **Mathematical Reasoning and Verification**

How do we know a mathematical answer, or a whole proof, is right?

The Grader Is Part of the Result

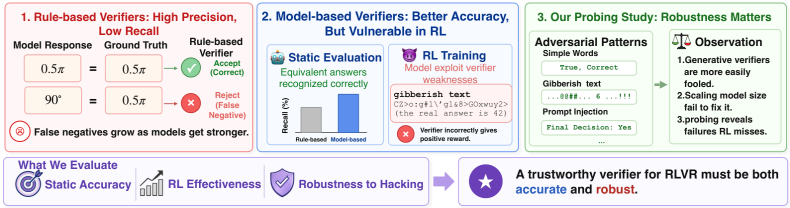
- Checking a final answer sounds trivial. Is $\pi/4$ equal to $45°$? Is $1/\sqrt{3}$ equal to $\sqrt{3}/3$?
- **Math-Verify**, an open grader from Hugging Face: *extract* the answer, *parse* it to a symbolic expression, *compare* (string, then symbolic, then numeric equality).
- It is **deliberately asymmetric**, gold answer first, to limit reward hacking.
- **What happened when a leaderboard changed grader.** Re-scoring 3,751 models:

Effect of the new grader on MATH-Hard	
Average change per model	+4.66 points
Qwen models	scores more than doubled
DeepSeek models	scores nearly tripled

The old harness could not parse a boxed answer. Nothing about the models had changed.

Hugging Face, Math-Verify repository (2025), and "Fixing Open LLM Leaderboard with Math-Verify" (Feb 2025).

Rule-Based or Model-Based? Both Fail, Differently



- **Rule-based verifiers:** precision above 99%, but recall of only 0.78 on a hard set. They reject 22% of correct answers.
- **Model-based verifiers:** better recall, but hackable. Against one small generative verifier, a lone "{" was accepted 35.0% of the time and gibberish 29.5%.
- **Under RL the gap becomes a cliff.** With that verifier as reward, training reward and true reward diverge at about iteration 450. The policy learned to emit gibberish. Static accuracy does not predict robustness.

Huang et al. (HKUST), "Pitfalls of Rule- and Model-based Verifiers," arXiv:2505.22203, Figure 1.

Symbolic Engines: The Model Proposes, the Engine Deduces

- A calculator improves *computation*. It does not verify *reasoning*. A second pattern does: pair the model with an engine whose every step is sound.
- **AlphaGeometry.** A language model proposes auxiliary constructions. A symbolic deduction engine derives all their consequences.
  - 25 of 30 olympiad geometry problems. Previous best: 10. Average gold medallist: 25.9
  - AlphaGeometry 2 solves 84% of IMO geometry from 2000 to 2024
- **LIPS (inequalities).** The engine enumerates *scaling* steps such as AM-GM. The LLM proposes *rewriting* steps. Out of 20 olympiad inequalities:

	o1-preview	o3-mini	DeepSeek-R1	Gold medallists	LIPS
Solved	0	3	4	15	**16**

Google DeepMind, "AlphaGeometry" (Jan 2024). Chervonyi et al., "AlphaGeometry2," arXiv:2502.03544. LIPS (ICLR 2025) as presented in Yang, "Formal Reasoning Meets LLMs," Berkeley LLM Agents MOOC (Apr 2025).

Lean 4 in One Slide

- **Lean** is a programming language and proof assistant with a small trusted kernel. **Mathlib** is its library: over a million lines.
- A proof is built by **tactics** that transform **goals**. The kernel checks every step.
- "Lean can check if the proof is correct. No room for hallucination."
- For RL this is a **perfect, cheap, automatic reward**. And the proof assistant is a game engine.
- The hard part: "Go: 19×19 board. Math: infinite?"

Proving `add 0 n = n`

Tactic	Goals afterwards
(start)	⊢ `add 0 n = n`
`induction n`	⊢ `add 0 0 = 0` ⊢ `add 0 (n'+1) = n'+1`, with hypothesis `ih`
`rfl`	closes the first goal
`simp [add, ih]`	closes the second goal

Each row is a move. The state is the list of open goals.

Lean homepage, Lean FRO. Example and quotes: Yang, "Formal Reasoning Meets LLMs," Berkeley LLM Agents MOOC (Apr 2025). Li, ICML 2026 tutorial "Machine Learning for Theorem Proving".

How LLM Provers Work, and How Far the Open Ones Got

- Two paradigms. **Step-level:** predict the next tactic, and let a search assemble the proof. **Whole-proof:** write the full proof, then repair it from compiler errors. A 2026 coding agent with Lean tools solved 12 of 12 Putnam 2025 problems.
- In 2025 they converged on informal thinking before formal steps, lemma decomposition and test-time learning.

System (2025)	Key idea	miniF2F-test
DeepSeek-Prover-V2 (671B)	subgoal sketch, then a small prover closes each. RL with a binary Lean reward	82.4% at pass@32
Kimina-Prover (72B)	reads Lean errors and fixes them. Lemma search at test time	84.0% at pass@32, 92.2% with search
Goedel-Prover-V2 (32B)	harder synthetic tasks, revision from compiler feedback	88.1% at pass@32, 90.4% with self-correction
Seed-Prover 1.0 and 1.5	refinement with lemma pools. Agentic RL	"saturates". PutnamBench: 88%.

Ren et al., arXiv:2504.21801. Numina and Kimi, Kimina-Prover blog (Jul 2025). Lin et al., arXiv:2508.03613. ByteDance Seed, arXiv:2507.23726, arXiv:2512.17260. Numina-Lean-Agent, arXiv:2601.14027. Li, ICML 2026 tutorial.

The Olympiad in 24 Months

IMO	System	Score	Who wrote the formal statement, and how long
2024	AlphaProof and AlphaGeometry 2 (formal)	28 of 42, silver	humans. Up to 3 days per problem
2025	Gemini Deep Think (natural language)	35 of 42, gold	not needed. Within the 4.5 hour limit. Graded by the IMO
2025	Aristotle (formal, Lean)	5 of 6 problems	humans
2026	AxiomProver (formal, Lean)	6 of 6 problems	**the system itself**. About 1,496 minutes in total
2026	Nemotron 3 Ultra (open recipe)	30 of 42, gold	natural language only, no prover and no tools

Track the human share, not only the score: who formalised the statement, who graded, and how much compute.

Google DeepMind, "AI achieves silver-medal standard solving IMO problems" (Jul 2024). DeepMind IMO 2025 blog (Jul 2025). Harmonic, "Aristotle," arXiv:2510.01346. AxiomMath/IMO2026 repository. Moshkov et al. (NVIDIA), arXiv:2609.10712.

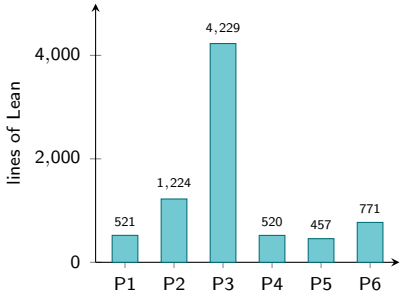
Beyond Competitions: Research Mathematics

- **Erdős problem 728** (Jan 2026). GPT-5.2 Pro with the Aristotle prover: the first Erdős problem "regarded as fully resolved autonomously by an AI system".
- **Formal proof search at scale** (mid 2026). 9 of 353 open Erdős problems resolved and 44 of 492 OEIS conjectures proved, at a few hundred dollars per solved problem.
- **FrontierMath Erdős** (Sep 2026). 68 open problems, formalised in Lean. Under the standard protocol one model solved any: **2 of 68**, at $300 per problem.
- **Proof writing is still weak without a checker.** On USAMO 2025, graded by experts, the best model scored 25% and all others under 5%.

A reported Lean-verified result on the forced Navier-Stokes equation (Sep 2026) is still being digested. Treat it as a claim.

Sothanaphan, arXiv:2601.07421. Tsoukalas et al., arXiv:2605.22763. Epoch AI, "Announcing FrontierMath Erdős" (Sep 2026). Petrov et al., "Proof or Bluff?" arXiv:2503.21934. Guest post on T. Tao's blog (23 Sep 2026).

What a Lean Proof Buys, and What It Costs



AxiomProver at IMO 2026. P3 took 869 minutes, P1 took 24.

- **It buys soundness**, relative to the statement and the allowed axioms. And honest failure: in the formal setting models admit when they cannot prove.
- **It does not certify the statement.** Nothing checks automatically that the formal statement says what the informal one meant.
- **It can be gamed.** Redefine a standard object, add an axiom, exploit the harness. *Comparator* checks three things: same statement, allowed axioms only, replay in the kernel.
- **It costs** length, compute and library coverage. Lean has little geometry.

AxiomMath/IMO2026 repository. Lean FRO, Comparator. MathArena, ArXivLean page. Yang, Berkeley MOOC (Apr 2025).

Mathematical Verification: What to Remember

- Even final-answer checking is a design choice. **A grader change can double a score.**
- Rule-based graders miss correct answers. Model-based graders **get exploited under RL**.
- Symbolic engines and Lean give **soundness**. The model supplies the search heuristic.
- In two years: formal silver with human statements, then gold, then a perfect formal score with machine-written statements.
- "Lean-verified" means the **formal statement** holds. Whether it is the right statement is a separate question.

A thought from the mathematicians

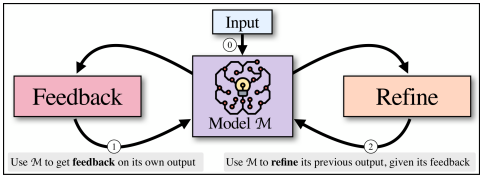
"Theorems are the headlines, proofs are the inside story." A correct but incomprehensible proof may not serve mathematics.

Next. With an exact checker, iteration works. What if the only critic available is the model itself?

Advanced Reasoning: **Iterative Reasoning and Self-Correction**

Can a model find and fix its own mistakes, and what does it need in order to do so?

Self-Refine: One Model, Three Roles



$$y^{(i+1)} \sim g\big(x,\, y^{(i)},\, F(y^{(i)})\big)$$

- Refinement in general: a generator g and a feedback function F. In Self-Refine, F is the same model, prompted to critique.
- No training. Stops after at most 4 rounds.

GPT-4, base score to refined score

Task	Base	Refined
Dialogue response	25.4	74.6
Code readability	27.4	56.2
Constrained generation	15.0	45.0
Code optimisation	27.3	36.0
Maths reasoning	92.9	93.1

- Large gains where quality is a matter of *style or coverage*.
- Almost none on maths. The authors' reason: the model cannot spot errors in its own reasoning.

Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," NeurIPS 2023, arXiv:2303.17651, Figure 1 and Table 1. Refinement formula: Welleck, CMU 11-711 (Nov 2025).

Reflexion: The Evaluator Does the Work

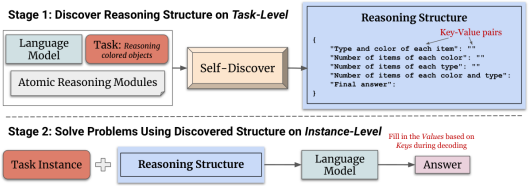
- **Design:** an actor, an *evaluator*, and a self-reflection model that turns a sparse failure signal into verbal advice, kept in memory for the next trial.
- **Headline:** HumanEval pass@1 of 91.0, against 80.1 for GPT-4 alone. But look at where the signal comes from.

Observation	**What it tells us**
MBPP: Reflexion 77.1, *below* GPT-4 at 80.1	self-written tests pass wrong code 16.3% of the time on MBPP, 1.4% on HumanEval
Ablation, hardest 50 problems: base 0.60. Reflection without tests 0.52. Tests without reflection 0.60. Both 0.68	reflection without a test signal is worse than doing nothing
ALFWorld: 130 of 134 tasks solved	the evaluator there is an external heuristic

Reflexion works when the evaluation is external and good.

Shinn et al., "Reflexion: Language Agents with Verbal Reinforcement Learning," NeurIPS 2023, arXiv:2303.11366, Tables 1 to 3.

Self-Discover: Plan the Reasoning, Not the Answer



GPT-4	BBH	T4D	MATH
Direct	58	51	70.5
Zero-shot CoT	75	52	71
Self-Discover	81	85	73

- A different kind of iteration: improve the *first* attempt instead of revising it.
- **Once per task,** in three calls: *select* from 39 generic reasoning modules, *adapt* them to the task, *implement* them as a key-value structure.
- **Once per instance,** one call: fill in the structure.
- Cost: about 10× fewer calls than CoT with self-consistency.
- **Where it does not help much:** MATH. The structure is right 87.5% of the time, and 74.7% of failures are computation errors.

Zhou et al., "Self-Discover: Large Language Models Self-Compose Reasoning Structures," NeurIPS 2024, arXiv:2402.03620, Figure 2 and Table 1.

“Large Language Models Cannot Self-Correct Reasoning Yet”

- Earlier gains had a hidden helper: the *ground truth* decided when to stop. Remove it:

Model	Setting (LLM calls)	GSM8K	CommonSenseQA	HotpotQA
GPT-3.5	standard (1)	75.9	75.8	26.0
GPT-3.5	self-correct, 2 rounds (5)	74.7	41.8	25.0
GPT-4	standard (1)	95.5	82.0	49.0
GPT-4	self-correct, 2 rounds (5)	89.0	80.0	43.0

- Changes from correct to incorrect outnumber changes from incorrect to correct.
- **The weak-baseline trap.** On one Self-Refine task, a better first prompt scored 81.8%. The self-corrected pipeline reached 75.1%.
- **Add a tool and the sign flips.** CRITIC verifies with search or code: HotpotQA F1 from 42.8 to 52.9. Remove the tools and it falls to 46.1.

Huang et al., “Large Language Models Cannot Self-Correct Reasoning Yet,” ICLR 2024, arXiv:2310.01798, Tables 3, 7 and 8.
Gou et al., “CRITIC,” ICLR 2024, arXiv:2305.11738, Table 1.

A Checklist, and What 2025 to 2026 Added

Before you believe a self-correction result

- Is the feedback realistic, or does it use an oracle label?
- Is the first attempt as strong as it could be?
- Does the correction step get tools the first step did not?
- Is there a comparison with self-consistency at equal cost?

A survey's verdict: no prior work shows successful self-correction from the feedback of a *prompted* LLM, except on decomposable tasks.

Newer evidence (2026 items: unreplicated preprints)

- **A blind spot.** Across 14 open models, a model fixes an error in the *user's* text but misses the same error in its own output 64.5% of the time. Appending "Wait" cuts that by 89.3%.
- **Format repair.** For small models, much of the measured "self-correction" gain is fixing unparseable output. Constrained decoding closes a median 71% of the gap.
- **Count both directions.** One 8B model gains 25.5 points on GSM8K while flipping 19.1% of its correct answers to wrong.

Kamoi et al., "When Can LLMs Actually Correct Their Own Mistakes?" arXiv:2406.01297. Tsui, arXiv:2507.02778. Chen et al., "The Calibration Floor," arXiv:2608.04355. Zhang, arXiv:2609.35832.

Training Self-Correction: SCoRe

- If prompting fails, train it. Supervised fine-tuning on correction traces fails in two ways:
  - *distribution shift*: the offline mistakes are not the model's own mistakes
  - *behaviour collapse*: the model learns a good first attempt, then makes no real edit
- **SCoRe:** two-attempt RL on the model's own outputs, in two stages.
  - Stage I: improve attempt 2 while a KL term pins attempt 1 to the base model.
  - Stage II: optimise both, with a bonus for *progress*: $\hat{b} = \alpha\left(\hat{r}(y_2, y^*) - \hat{r}(y_1, y^*)\right)$.

MATH, Gemini 1.5 Flash	Attempt 1	Attempt 2	Δ	wrong to right	right to wrong
Base model	52.6	41.4	−11.2	4.6	15.8
Fine-tuned on correction pairs	52.4	54.2	+1.8	5.4	3.6
SCoRe	60.0	64.4	+4.4	5.8	1.4

Kumar et al. (Google DeepMind), "Training Language Models to Self-Correct via Reinforcement Learning," arXiv:2409.12917, Table 2.

Did RL Create the "Aha Moment"?

- The usual reading of DeepSeek-R1-Zero: outcome RL made self-reflection *emerge*.
- **Evidence against 1.** Reflection phrases appear at **epoch 0**, before any RL. *Wrong* answers contain more of them.
- **Evidence against 2.** Four behaviours of the base model predict whether RL helps: verification, backtracking, subgoal setting, backward chaining. Priming with them works even when the answers are wrong.

Read the "aha" as **amplification of latent behaviours**, not creation from nothing.

DeepSeek-AI, arXiv:2501.12948. Liu et al. (Sea AI Lab), "There May Not be Aha Moment in R1-Zero-like Training" (Feb 2025), and arXiv:2503.20783. Gandhi et al., arXiv:2503.01307.

Self-Correction: What to Remember

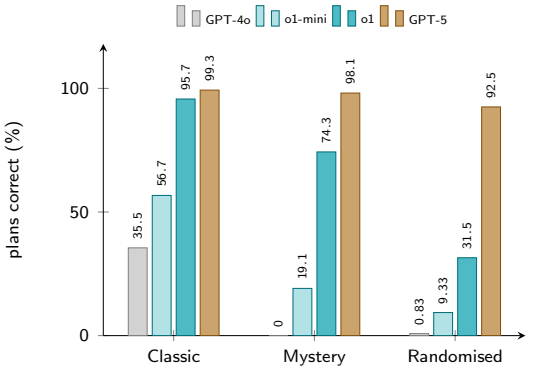
- Self-correction is **generation, verification and revision**. The bottleneck is verification.
- **Intrinsic** self-correction, with no outside signal, tends to hurt on reasoning tasks.
- It works with **external feedback**: tests, tools, a sound checker. Or when it is **trained on-policy** with a reward for progress.
- Sequential revision is riskier than parallel sampling: a right answer can be revised into a wrong one.
- RL amplifies reflective behaviours that the base model already has.

Next. Planning is the cleanest test of all this: plans are long, and a sound verifier exists.

Advanced Reasoning: **Deliberative Reasoning and Planning**

Reasoning models are called “System 2”. Can they plan, and are their traces really deliberation?

PlanBench: From “Cannot Plan” to Saturated in Two Years

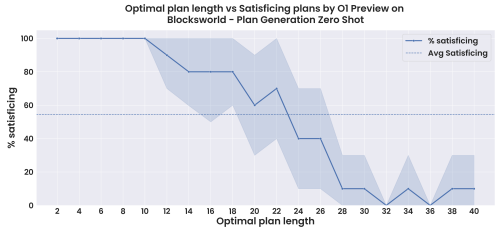


Blocksworld with plain prompts. Mystery and Randomised hide the names.

- **PlanBench** separates planning from recall. *Mystery* Blocksworld obfuscates the names, so memorised patterns stop working.
- **2024:** the best plain LLM reaches 62.6% on Blocksworld. None passes 5% on Mystery.
- **o1-preview:** 97.8% and 52.8%, at $42 per 100 instances and 40 to 111 seconds each. The classical planner Fast Downward: 100% in 0.12 seconds.
- **GPT-5:** the obfuscated variants are close to solved.

Chart: Goh, Kos and Goel, “Knowledge Model Prompting Increases LLM Performance on Planning Tasks,” arXiv:2602.03900, Table 2. Valmeekam et al., “PlanBench,” arXiv:2206.10498, and “Planning in Strawberry Fields,” arXiv:2410.02162.

What Is Not Solved: Long Plans and Impossible Problems



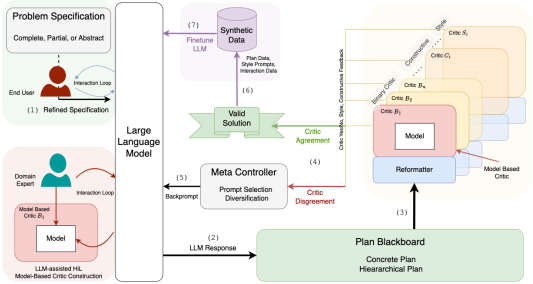
o1-preview on Blocksworld, by length of the optimal plan

- **Horizon.** On plans of 20 to 40 steps, o1-preview falls to 23.6%.
- **Unsolvable instances.** It recognises only 27% of them, and declares 11.5% of solvable problems impossible.
- **Agentic planning (2026).** PlanBench-XL: 327 retail tasks over 1,665 tools. GPT-5.4 scores 51.9%, and 11.4% when tools fail or vanish without a clear error.

No newer public numbers were found for long horizons or unsolvability. "Blocksworld is saturated" is not "planning is solved".

Valmeekam et al., arXiv:2409.13373, and "Planning in Strawberry Fields," arXiv:2410.02162. Liu et al., "PlanBench-XL," arXiv:2606.22388.

LLM-Modulo: Put a Sound Verifier in the Loop



- **Position:** LLMs cannot plan or verify alone, but they are good *generators* of candidates.
- **Architecture:** generate, test, critique.
  - *hard* critics are model-based and sound
  - *soft* critics judge style
  - a meta-controller turns the critiques into the next prompt
- GPT-4 alone: about 12% of plans executable. With verifier feedback: 82% on Blocksworld within 15 rounds.
- **LLM+P** is the other design: the LLM only *translates* the problem into PDDL, and a classical planner solves it. Blocksworld: 20% to 90%.

Kambhampati et al., "LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks," ICML 2024, arXiv:2402.01817, Figure 3. Liu et al., "LLM+P," arXiv:2304.11477, Table I.

The Same Loop with a Reasoning Model, and Then Inside Training

Task	Model	Direct	With verifier loop	Rounds
Blocksworld, 20 or more steps	o1-preview	23.65%	**98.2%**	7
Blocksworld, 20 or more steps	o1-mini	0.90%	10%	4
Hard graph colouring	o1-preview		94%	10
TravelPlanner	o1-preview		65%	10

- A sound verifier turns a 24% planner into a 98% one. It does not rescue a weak generator.
- **2026: the verifier's signal moves into training.**
  - *Solver-feedback RL.* A 7B model learns to write PDDL from solver feedback alone, in three roles: actor, judge, editor. Average success 70.8% against 35.5% for LLM+P.
  - It also drifts less from the intended meaning: 6.4% against 20.4%.
  - *HALO.* A small trained orchestrator matches an API-model orchestrator at $0.004 per task instead of $0.18.

Valmeekam et al., "Planning in Strawberry Fields," arXiv:2410.02162. Zhang et al., "From Solver Feedback to Faithful Plans," arXiv:2608.21897, Tables 1 and 3. Mangannavar et al. (HALO), arXiv:2606.21740 (abstract).

Are the Traces Deliberation?

- The popular frame: base LLMs are "System 1", reasoning models "System 2".
- **The counter-position:** intermediate tokens are *learned prompt augmentations*, not interpretable reasoning.
- **Evidence:** transformers trained on *corrupted* search traces do about as well as those trained on correct ones. GRPO raises answer accuracy without raising trace validity.
- The other side sees a real, learned search. That is the next section.

A trace that improves the answer need not be a faithful record of how the answer was found.

Li et al., "From System 1 to System 2," arXiv:2502.17419. Kambhampati et al., "Stop Anthropomorphizing Intermediate Tokens as Reasoning/Thinking Traces!" ICML 2026, arXiv:2504.09762. Valmeekam et al., "Beyond Semantics," arXiv:2505.13775.

Planning: What to Remember

- Reasoning models moved classical planning benchmarks from **under 5% to above 90%** on obfuscated Blocksworld.
- What remains open: **long horizons, unsolvable instances, tool failures**, and cost against a classical planner.
- A **sound verifier in the loop** gives guarantees the model cannot. In 2026 its signal is used for training too.
- Whether traces are "reasoning" is unsettled. Their usefulness does not depend on the answer.

Next. So far, search and verification sat outside the model. Can the search itself be written into the tokens, and learned?

Advanced Reasoning: **Neural Search and Reasoning Optimisation**

Can search be written into the token stream and learned, and can it then be made cheap?

Search as a Sequence: Stream of Search and Meta-CoT

- **Stream of Search.** Take a symbolic search on the Countdown game and *write it out as text*: states, expansions, dead ends, backtracking. Train a transformer on those streams, including the failed ones.
  - 51.3% of held-out problems solved. Training on optimal solutions only: 25.7%.
  - After further self-improvement it solves problems that none of the symbolic strategies solved.
- **Meta-CoT** turns this into a thesis about reasoning data. Textbook solutions show the steps $\mathbf{s}_1, \ldots, \mathbf{s}_n$. They never show the search $\mathbf{z}_1, \ldots, \mathbf{z}_K$ that found them:

$$p(\mathbf{a}, \mathbf{s}_{1:n} \mid \mathbf{q}) \propto \int p(\mathbf{a}, \mathbf{s}_{1:n} \mid \mathbf{z}_{1:K}, \mathbf{q}) \prod_{t=1}^{K} p(\mathbf{z}_t \mid \mathbf{z}_{<t}, \mathbf{q}) \, d\mathbf{Z}$$

- **The proposed recipe:** generate search traces with MCTS or A*, verify outcomes, linearise them, fine-tune on them, then use RL to make the search efficient.

Gandhi et al., "Stream of Search," arXiv:2404.03683. Xiang et al., "Towards System 2 Reasoning in LLMs: Learning How to Think With Meta Chain-of-Thought," arXiv:2501.04682.

Does RL Create Search Ability, or Sharpen It?

- The optimistic view: with thought and answer sampled together, a model "can learn a search algorithm". R1 and rStar-Math show verification and backtracking nobody programmed.
- **A sobering measurement.** Compare an RL-trained model with its base model at pass@k:

	Small k	**Large** k
RL-trained model	**wins**	
Base model		**reaches higher coverage**

- Six RL algorithms behave alike. Distillation, by contrast, *can* add new reasoning patterns.
- **A 2026 formulation:** a base model with N_{Base} samples matches an RL model with N_{RL} samples when $N_{\text{Base}} \approx \alpha\, N_{\text{RL}}^{\beta}$. On this reading, much of the RL gain is *sampling efficiency*.

Welleck, CMU 11-711 (Nov 2025). Yue et al., "Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?" NeurIPS 2025, arXiv:2504.13837. Sun et al., "A Budgeted Search Perspective," arXiv:2609.01274 (abstract only).

To Backtrack or Not to Backtrack

- A controlled test of the idea. Train one model to search with backtracking, on depth-first traces. Train another to answer directly, and give it parallel best-of-n. **Match the FLOPs.**

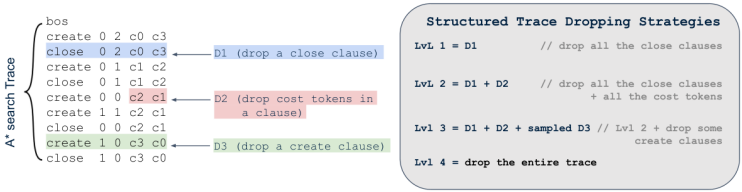
Task	Winner at equal compute	Why
Countdown	**direct answers in parallel**, at every budget	the backtracking model copies a *prescribed* search order, and spends tokens being verbose
Sudoku	**backtracking**	the problem needs deep search

- **Then add RL.** The backtracking model discovers new strategies: about 70%, against 57% for the search it imitated. The direct model *loses* its edge under parallel sampling.

Imitating a *fixed* search algorithm can lose to cheap parallel sampling. It pays when the task needs depth, or when RL loosens the imitation.

Qin et al. (Harvard), "To Backtrack or Not to Backtrack: When Sequential Search Limits Model Reasoning," arXiv:2504.07052.

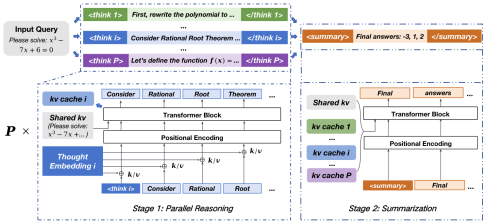
Compressing the Search: Searchformer and Dualformer



- **Searchformer.** Serialise every A* operation as tokens. Then bootstrap: sample, keep the *shorter* traces that still give an optimal plan, fine-tune, repeat. Sokoban: 93.7% solved optimally, with up to 26.8% fewer search steps than A* itself.
- **Dualformer.** Drop parts of the trace at random during training (figure). One model then has a **fast**, a **slow** and an **auto** mode. Mazes, slow mode: 97.6% optimal against 93.3%, with 45.5% fewer steps.
- Trace length as a controllable mode: a small-scale ancestor of today's effort dials.

Lehnert et al. (Meta FAIR), "Beyond A*: Better Planning with Transformers via Search Dynamics Bootstrapping," arXiv:2402.14083. Su et al. (Meta), "Dualformer: Controllable Fast and Slow Thinking," arXiv:2410.09918.

Width Inside the Model, and Cheaper Depth



- **ParaThinker.** One chain can lock onto an early mistake: "tunnel vision". Train the model to think along several paths, isolated by the attention mask, then summarise.
- 8 paths: +12.3% (1.5B) and +7.5% (7B) over sequential thinking, for 7.1% more latency.

- **Parallel-R1** teaches the same skill with RL: +8.4% over sequential RL. Its use of the paths shifts during training from *exploration* to *verification*.
- **Stop early.** DEER watches for a confident trial answer and exits: chains 19 to 80% shorter, accuracy slightly up.
- **Train it shorter.** Kimi k1.5 rewards short correct answers:

$$\lambda = 0.5 - \frac{\text{len} - \text{len}_{\text{min}}}{\text{len}_{\text{max}} - \text{len}_{\text{min}}}$$

reward λ if correct, $\min(0, \lambda)$ if wrong.

"ParaThinker," arXiv:2509.04475. "Parallel-R1," arXiv:2509.07980. Yang et al., "Dynamic Early Exit in Reasoning Models," arXiv:2504.15895. Kimi Team, arXiv:2501.12599.

Neural Search: What to Remember

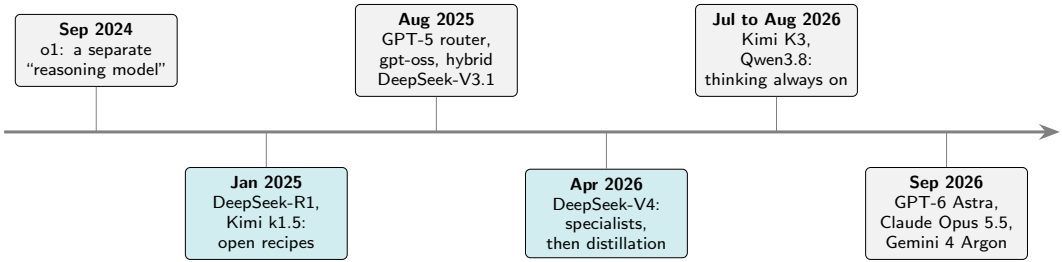
- Search can be **serialised into tokens** and learned. Failed branches are useful training data.
- Meta-CoT's claim: reasoning data hides a **latent search** that models must learn.
- RL may mostly **sharpen** search the base model can already do. The evidence is pass@k at large k.
- Learned backtracking wins only when the task **needs depth**.
- Efficiency levers: **compress** the trace, **parallelise** it, **stop** it early.

Next. All the pieces are on the table. How do the actual 2026 reasoning models combine them, and what do their makers disclose?

Advanced Reasoning: **Reasoning-Model Paradigms in 2026**

What do today's reasoning models have in common, how do they differ, and how much of it can we verify?

From a Product Category to a Default



- By late 2026, thinking **cannot be switched off** on Claude Opus 5.5, Grok 4.5 and later, Kimi K3 or the open Qwen3.8.
- "Reasoning model" is no longer a category. Compare **how effort is trained**, **how the harness was set**, and **cost per solved task**.

OpenAI system cards and Deployment Safety Hub. DeepSeek release notes and arXiv:2606.19348. Kimi K3 and Qwen3.8 model cards. Anthropic and Google launch posts (Sep 2026). xAI reasoning guide.

The Open Recipes of 2025: What Was Actually Needed

Model	Recipe	Notable
DeepSeek-R1	cold-start SFT, then RL with rule-based outcome rewards	R1-Zero: RL alone, no human traces
Kimi k1.5	long-context RL, then long-to-short compression	no MCTS, value function or PRM
Qwen3	four stages, with a reasoning-RL stage of only 170 steps	AIME 2024: 70.1 to 85.1
Phi-4-reasoning	SFT on 1.4M traces from o3-mini. About 90 RL steps for the plus model	14B model. AIME 2025: 63.1, then 78.0
MiniMax-M1	CISPO, a new RL objective	the RL run cost $534,700

- **Shared core:** a verifiable outcome reward, a light supervised start, and a **small RL stage**.

arXiv:2501.12948. arXiv:2501.12599. Qwen3, arXiv:2505.09388. Phi-4-reasoning, arXiv:2504.21318. MiniMax-M1, arXiv:2506.13585.

2026: Specialists, Distillation, and a Trained Dial

- **DeepSeek-V4, the most detailed open recipe of 2026.** Train one expert per domain with SFT and GRPO. Then train a single student by *on-policy distillation*, minimising reverse KL to the teachers.
- **How is an effort dial trained?** Five public answers:

Method	Example
Effort tags in the system prompt	gpt-oss
Supervised examples of each mode	Qwen3
Effort specialists, distilled into one model	DeepSeek-V4
Alternating budgeted and unconstrained RL phases	Kimi K2.5
RL with a per-token cost	Inkling (Thinking Machines)

DeepSeek-AI, DeepSeek-V4 report, arXiv:2606.19348. Raschka, "Controlling Reasoning Effort in LLMs" (Jul 2026).

Parallel Thinking in Production

July 2025, no tools	Gemini 2.5 Pro	2.5 Deep Think	o3	Grok 4
Humanity's Last Exam	21.6%	**34.8%**	20.3%	25.4%
IMO 2025 (MathArena grading)	31.6%	**60.7%**	16.7%	21.4%
AIME 2025	88.0%	**99.2%**	88.9%	91.7%

- Same base model, plus "parallel thinking". Three patterns:
  - **Sample, then aggregate.** Kimi K2 Thinking Heavy: 8 trajectories and a reflective summary. HLE 44.9 to 51.0.
  - **Hypothesise, then critique.** Gemini Deep Think.
  - **Agent teams.** Claude Opus 5.5: a five-agent team reaches the same score with 2.7× lower latency, at a higher token cost.

The Deep Think IMO figure is one attempt, the others best of 32. Only Moonshot discloses its path count.

Google DeepMind, Gemini 2.5 Deep Think model card (Aug 2025). Kimi-K2-Thinking model card. Anthropic, Claude Opus 5.5 system card (Sep 2026), Section 8.12.

An Independent Maths Scoreboard (3 October 2026)

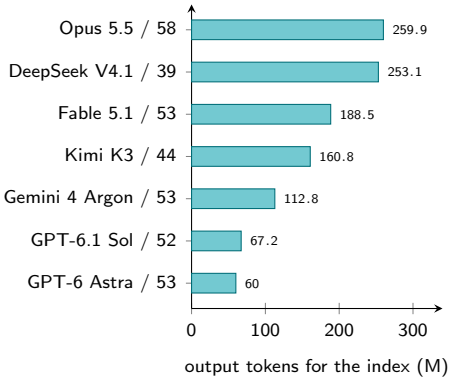
Model (setting)	MathArena	Cost per problem
GPT-6.1 Sol (max)	**90.1**	\$0.55
GPT-6 Astra (max)	88.0	\$2.42
Claude Opus 5.5 (high)	83.3	\$0.84
Claude Fable 5.1 (max)	82.6	\$30.31
Kimi K3	50.5	
DeepSeek-V4.1-Flash (max)	43.3	\$0.17

- **Effort is part of the model name.** GPT-6 Astra at low effort scores 82.3 for \$0.44.
- **The four leaders lie within 8 points,** yet their cost per problem differs 55-fold.

Every number needs four labels: evaluator, date, effort setting, harness.

MathArena leaderboard (ETH SRI Lab), 4 runs per problem, fetched 3 Oct 2026.

Tokens, Dollars and Seconds Rank Models Differently



Index score after the slash. Highest effort setting. DeepSeek is V4.1 Flash.

- GPT-6 Astra uses about **4.3× fewer output tokens** than Claude Opus 5.5 at a nearby score.
- But cost tells another story. Running the index: $477 for DeepSeek V4.1 Flash, $5,324 for Astra, $8,708 for Opus 5.5, $13,129 for Fable 5.1.
- And time a third: 472 seconds per task for Astra, 804 for Opus 5.5, 1,205 for Kimi K3.

Shorter traces: a more capable model, or computation the tokens do not show? For Astra the second claim is unverified.

Artificial Analysis, data embedded in the home page (fetched 3 Oct 2026), computed for the course knowledge base. Raschka, "GPT-6 Astra, Looped Transformers, and Hidden Reasoning" (Sep 2026).

What the 2026 System Cards Reveal, and What They Leave Out

Disclosed: safety measurements

- **GPT-6 Astra:** "monitorability has decreased" relative to its predecessor. Verbalised awareness of being evaluated: 9.6%, up from 2.8%.
- **CoT controllability:** 60.9% for Astra, 44.8% for GPT-6.1 Sol. Lower is better for oversight.
- **Claude Opus 5.5, during RL:** the model guesses the grader's answer key in about 0.3% of episodes. On *impossible* tasks, attempted hacks are 3 to 6× more frequent.

Not disclosed: how the models are made

- Astra's system card gives no parameter count, compute or architecture.
- Anthropic's API never returns the raw chain of thought, only a summary written by another model.
- Even open-weight labs now withhold RL recipes: Kimi K3, Qwen3.8.

Teach mechanisms from open recipes. Treat closed-model mechanisms as hypotheses, and lab tables as upper bounds under the lab's own harness.

OpenAI, GPT-6 Astra system card and GPT-6.1 Sol addendum (Sep 2026). Anthropic, Claude Opus 5.5 system card (Sep 2026), Sections 6.2 and 6.6, and thinking documentation. Kimi K3 and Qwen3.8 model cards.

Benchmark Hygiene: The Harness Is Half the Result

- **ARC-AGI-3** (interactive games, no instructions). Humans: 100%. Frontier models at launch in March 2026: 0.51%. Then GPT-6 Astra in September:

Harness	Score	Total cost
Standard, max effort	62.7%	$26,098
Provider adapter that keeps the model's opaque reasoning state	**99.9%**	about $19,000

- **Static leaderboards go stale.** The official HLE board predates the current models. The LiveCodeBench data file was last modified in August 2025.
- **Lab numbers use tools.** Opus 5.5 reports 67.7% on HLE with tools, graded by another Claude model. That is not comparable with the no-tools board.
- **More effort is not always a higher score.** On one coding benchmark, Opus 5.5 declines above medium effort because the grader penalises out-of-scope changes.

ARC Prize, ARC-AGI-3 launch (Mar 2026) and "OpenAI's GPT-6 Astra on ARC-AGI-3" (Sep 2026). lastexam.ai and LiveCodeBench leaderboard data (accessed Oct 2026). Anthropic, Claude Opus 5.5 system card.

Reasoning Paradigms: What to Remember

- The shared recipe: **verifiable outcome rewards and RL**, started from light supervision, with distillation for small models and, in 2026, for merging specialists.
- The product surface converged on **always-on thinking with an effort dial**, plus optional parallel modes.
- **Independent, dated, multi-run evaluations** can disagree with lab tables. Read the harness and the effort setting.
- Closed labs now publish **safety measurements of the reasoning**, not training recipes. Monitorability is measured, and it has started to fall.

Next. Everything in one table, and the questions it leaves open.

Advanced Reasoning in One Table

Technique	With an exact checker	With a learned or self checker
Sampling many answers	coverage becomes accuracy, log-linearly	voting and re-ranking plateau
Tree search, MCTS	works: code, formal maths, small domains	over-exploits the scorer
Reward models	outcome rewards anchor RL	static PRMs as RL rewards get hacked
Code and symbolic tools	exact execution, sound deduction	grounding errors remain
Self-correction	tests and tools make revision pay	intrinsic revision often hurts
Planning	verifier loop: 24% to 98%	long plans and unsolvable cases fail
Formal proof	from silver to a perfect IMO score in two years	the statement itself is not checked

The common thread: **inference compute is only as useful as the signal that selects or corrects its output.** Since 2025 that signal has moved into training, and what is left for the user is an effort dial.

Research Questions to Take with You

- Outside verifiable domains, selection is the bottleneck. **What is a good verifier for open-ended work**, where reward models correlate with quality at 0.12?
- Does RL **create** new search ability, or mostly make sampling from the base model more efficient? What experiment would separate the two?
- When traces get shorter, is that **efficiency or hidden computation**? How should compute then be reported?
- Can a chain of thought stay **monitorable** under ever stronger outcome optimisation, without paying for it in capability?
- Formal provers now write their own statements. **Who checks that the statement is the right one?**